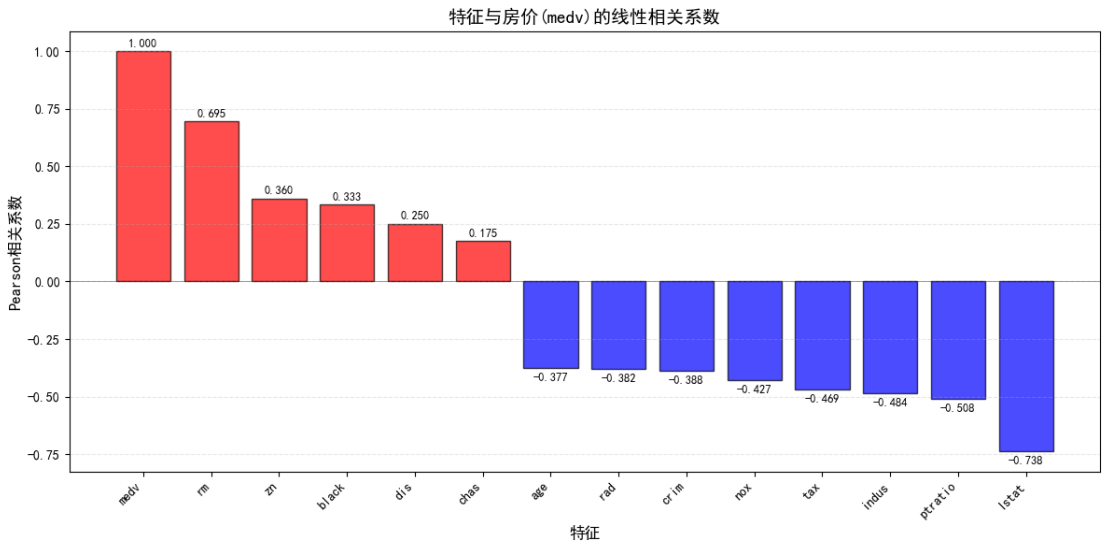
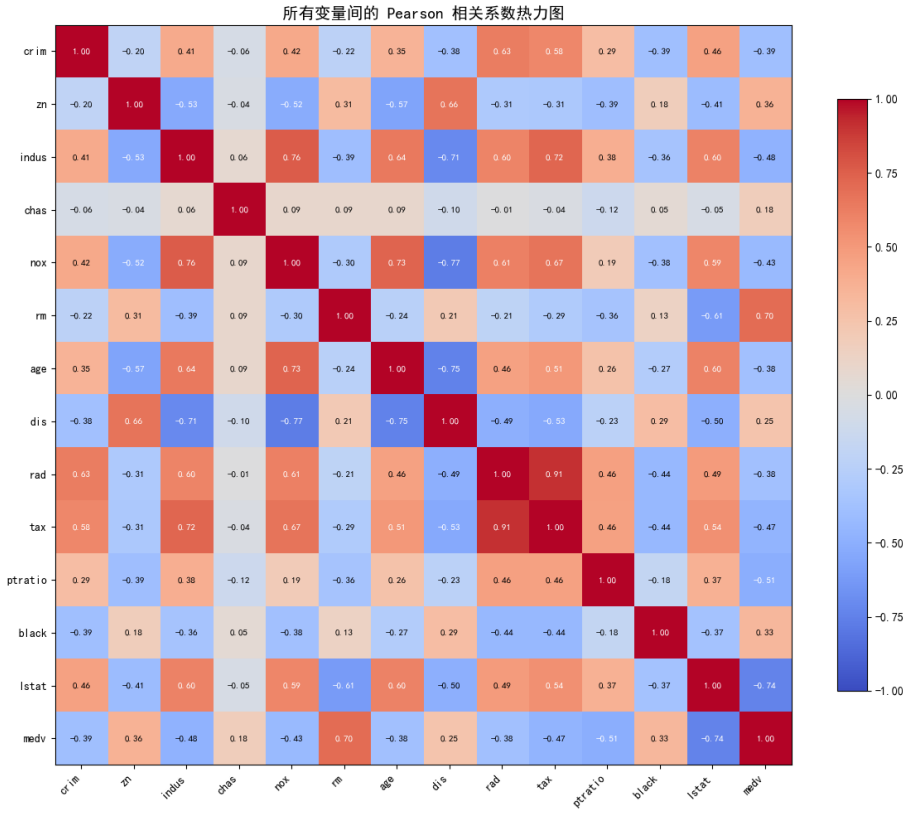
波士顿房价预测

1. 数据预处理

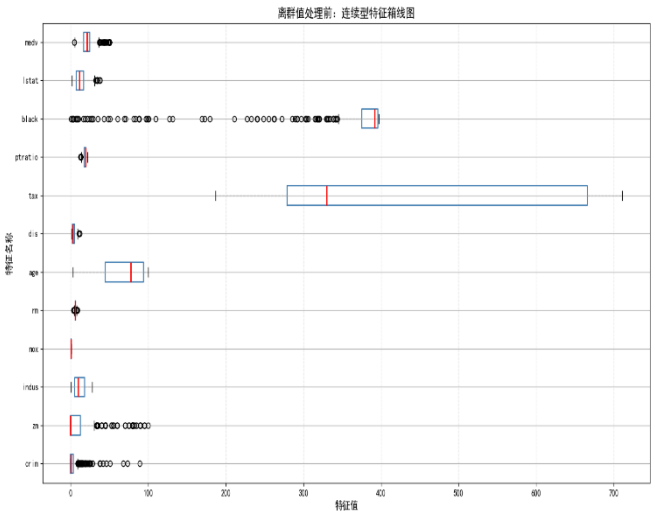
数据集无缺失值。各特征间的相关系数如下图：

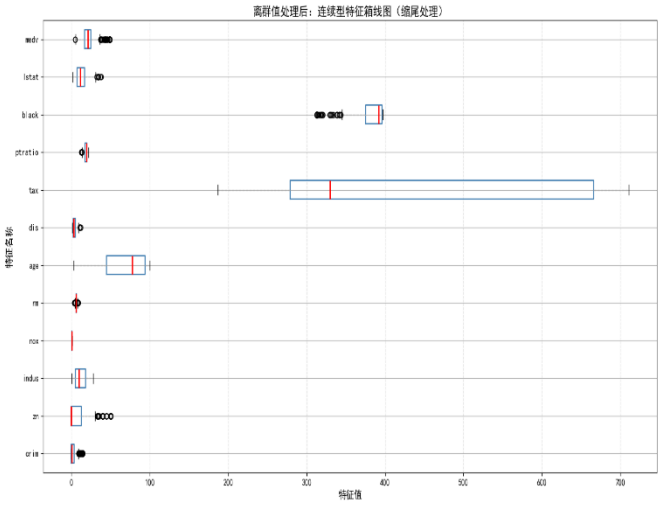




可以看出 rm 和 lstat 对房价的影响较大，特别注意 chas 特征虽然相关系数较小，主要是因为它是一个二值变量，一般生活中临河与否会显著地影响房价，我们将在回归之后查看 chas 变量的系数来验证这一点。

利用四分位数方法（IQR）判别连续型特征的离群值，占比 2.10%，为了使用线性回归模型，对数据进行缩尾处理，将离群点的值改为边界值，由箱线图可以看到，离群值处理前后离群值比例显著降低。



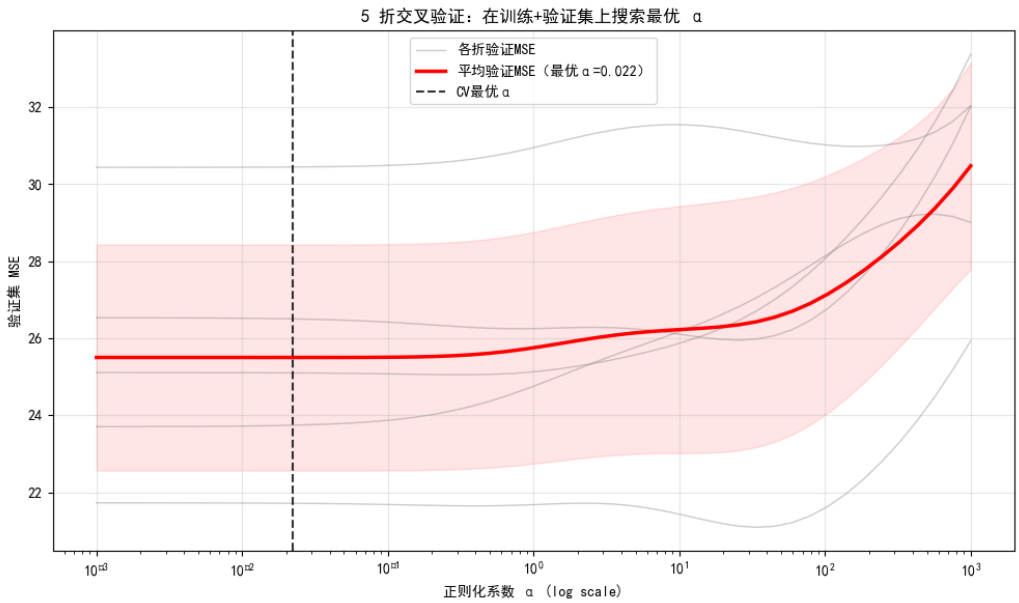


按 8：2 比例划分训练集和验证集，由于 chas=1 的数量较少，划分时强制令训练集和验证集中含有等比例的 chas=1 的数据。

2. 算法实现与结果

2.1 回归模型

选取加入正则项（岭回归）的线性回归模型，利用 sklearn 中的 `GridSearchCV` 函数对训练集进行**五折交叉验证**以选取最优的正则化系数，得到最优系数为 0.0222，说明数据复共线性较弱，几乎不需要正则化。



训练+验证集上 MSE = 22.6721，R^2 = 0.7270；在测试集上的最终结果为：MSE = 20.5174，R^2 = 0.7457。模型泛化能力较好。

在进行预测时，由于进行了标准化，对于第 i 个连续特征x_i需要依据该特征在训练集上的均值μ_i和标准差σ_i进行变换：$x_i' = \frac{x_i - \mu_i}{\sigma_i}$，非连续变量保持不变，根据最后得到的回归模型系数，有：

$$\text{medv} = -17.4904 \cdot \text{nox}' + 3.2919 \cdot \text{rm}' + 2.4239 \cdot \text{chas}'$$

$$-1.2782 \cdot \text{dis}' - 1.0209 \cdot \text{ptratio}' - 0.5804 \cdot \text{lstat}'$$

$$+0.3177 \cdot \text{rad}' - 0.2649 \cdot \text{crim}' + 0.0510 \cdot \text{zn}' + 0.0434 \cdot \text{indus}'$$

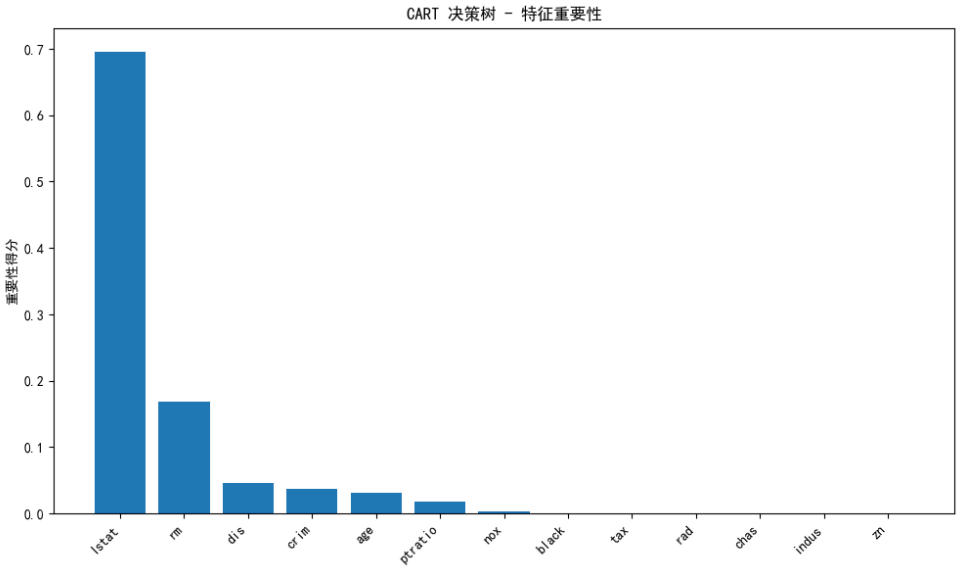
标准化后连续型变量的系数可以互相比较，可以看到 lstat 特征的负影响最大，rm 特征正影响次之，和相关性分析大致一样，chas 的系数为 1.6509，表明同等条件下在河岸边的房子比不在河岸附近的平均高 1650.9 美元。另外，在后续年份进行预测时，要根据年利率 r 计算现值：

$$\text{PV}_t(\text{medv}) = \text{medv} \times (1 + r)^{t-1978}$$

2.2 CART 决策树回归模型

另外对于 CART 决策树回归模型仍进行缺失值处理、离群值检测与训练集-测试集划分和超参数调优，但由于树模型对尺度不敏感，不进行标准化。

得到在训练集上 MSE = 6.4178， R^2 = 0.9231；测试集：测试集 MSE = 24.0974，R^2 = 0.6899，模型泛化能力较差，严重过拟合。



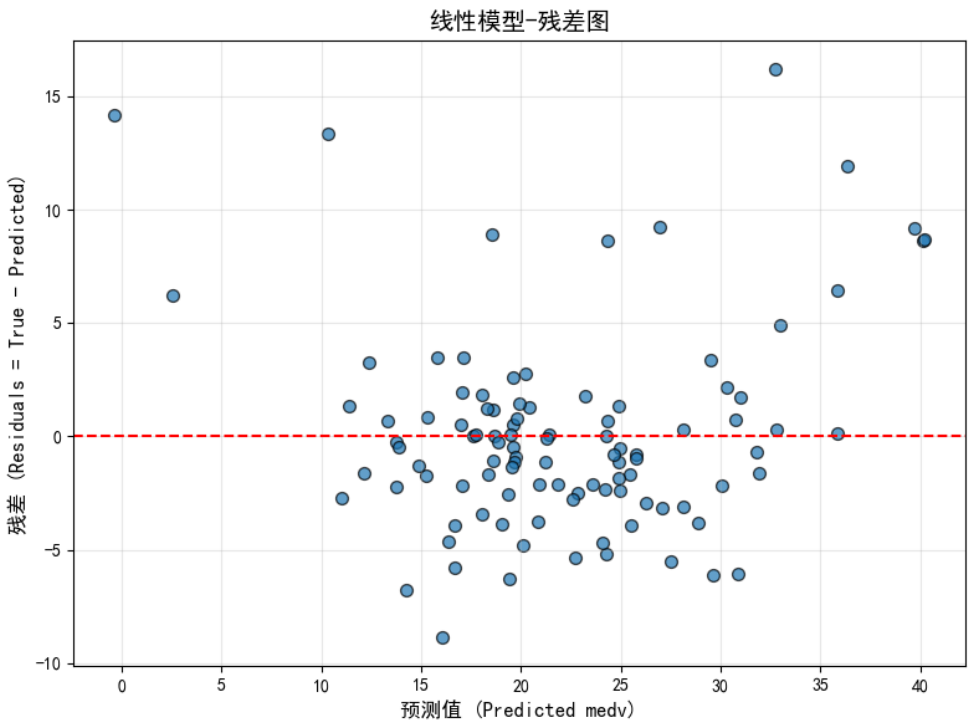
观察模型中各个特征重要性，lstat 和 rm 依然是主要的影响特征，而相关性较弱的特征影响几乎为 0。

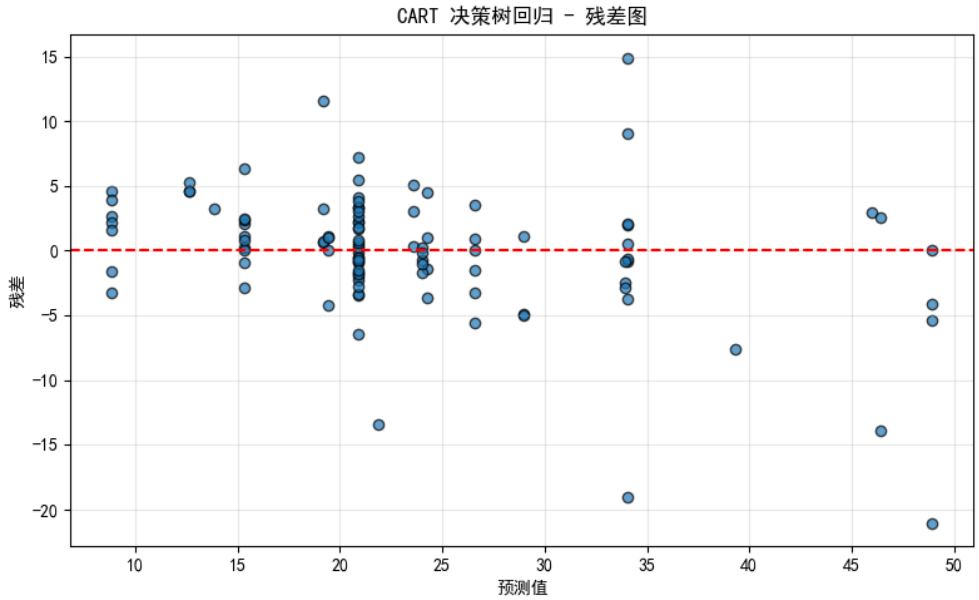
2.3 模型对比

对比两个模型，不难发现线性回归在准确率和泛化能力方面都要优于回归树模型，主要原因在于回归树本质需要将一个连续的变量离散化，用一个值代表整个区间，并不适合用于预测连续型的变量。

模型	训练 R^2	测试 R^2	泛化差距	可解释性	适用场景
岭回归	0.782	0.746	小	高（系数明确）	需解释变量影响
CART 树	0.923	0.690	大	中（特征重要性）	仅需预测，关系非线性

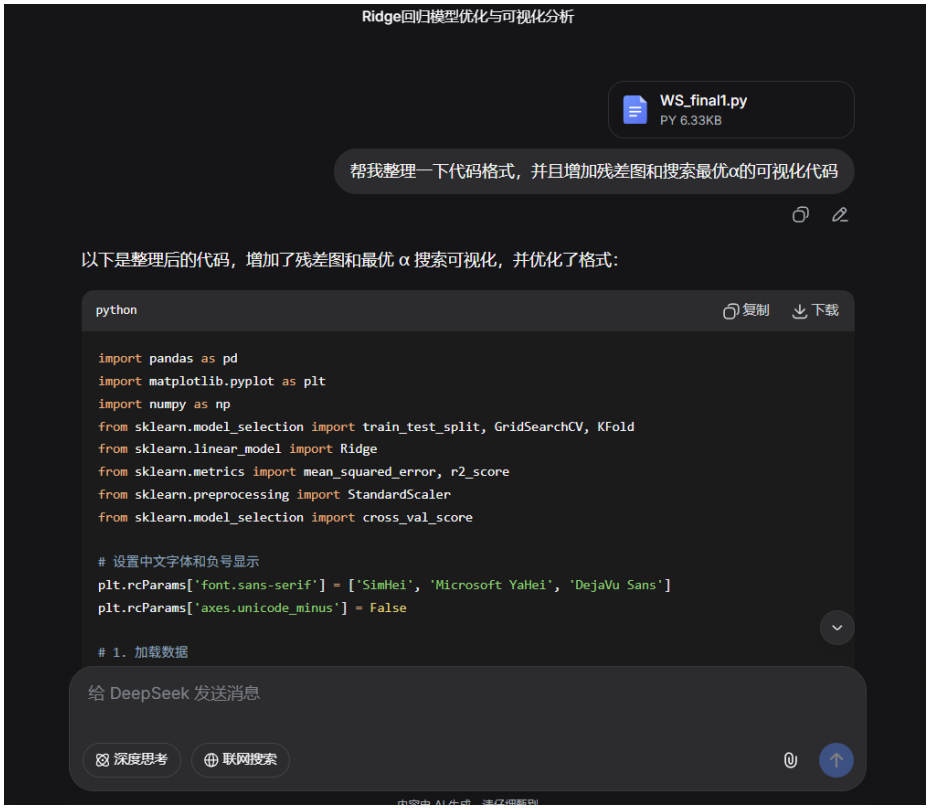
另外，从残差图可以看出，线性回归模型在预测的中间位置残差较小、接近零、满足线性回归假设，在较小和较大预测值位置残差较大，甚至有小于 0 的异常预测；决策树回归在各个预测区间均有残差较大的点。

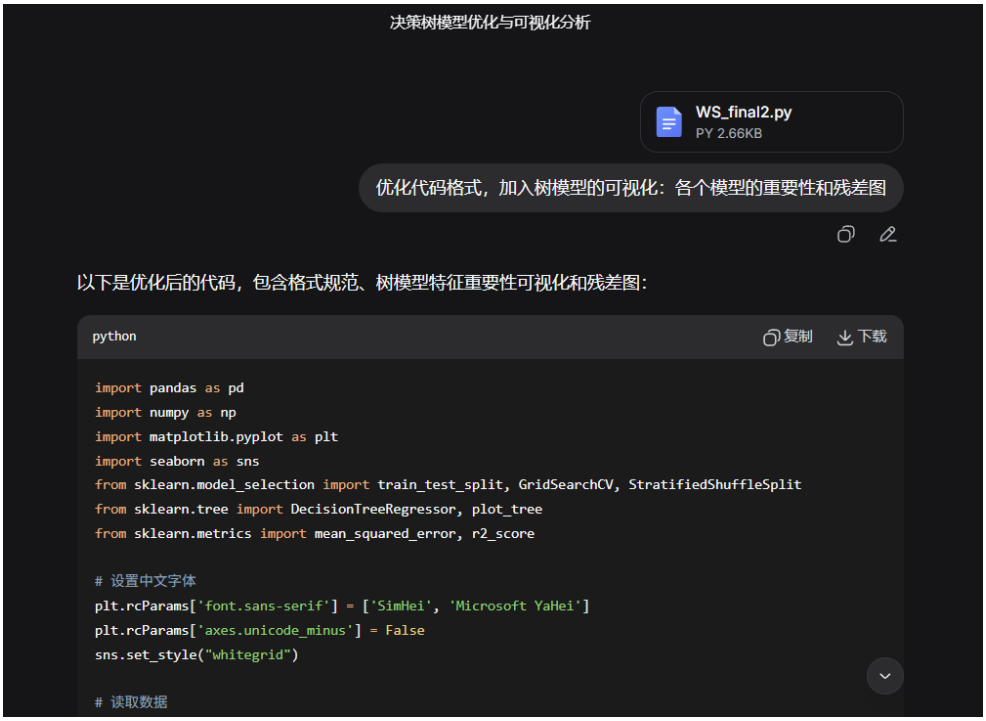

线性模型-残差图
残差 (Residuals = True - Predicted)
预测值 (Predicted medv)


CART 决策树回归 - 残差图
残差
预测值

AI 使用情况

主要用于优化代码格式和可视化





代码

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV,
KFold
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu
Sans']
plt.rcParams['axes.unicode_minus'] = False

filename = r"C:\Users\29414\Desktop\数据挖掘作业\波士顿
\boston_housing.csv"
df = pd.read_csv(filename)

print("缺失值检查")
missing_count = df.isnull().sum()
if missing_count.sum() == 0:
    print("无缺失值")
    print(f"数据集形状: {df.shape}")

print("\n特征与目标变量(medv)的相关系数")
correlations = df.corr()['medv'].sort_values(ascending=False)
print(correlations.to_string())

plt.figure(figsize=(12, 6))
colors = ['red' if x >= 0 else 'blue' for x in correlations.values]
bars = plt.bar(correlations.index, correlations.values, color=colors,
edgecolor='black', alpha=0.7)
for bar, value in zip(bars, correlations.values):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2,
             height + (0.01 if height >= 0 else -0.02),
             f'{value:.3f}',
             ha='center', va='bottom' if height >= 0 else 'top',
fontsize=9)
plt.axhline(y=0, color='black', linestyle='-', linewidth=0.5, alpha=0.5)
plt.title('特征与房价(medv)的线性相关系数', fontsize=14)
plt.xlabel('特征', fontsize=12)
plt.ylabel('Pearson 相关系数', fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3, linestyle='--')
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 10))
corr_matrix = df.corr()
im = plt.imshow(corr_matrix, cmap='coolwarm', aspect='auto', vmin=-1,
vmax=1)
```

```
plt.colorbar(im, shrink=0.8)

plt.xticks(ticks=np.arange(len(corr_matrix.columns)),
labels=corr_matrix.columns, rotation=45, ha='right')
plt.yticks(ticks=np.arange(len(corr_matrix.columns)),
labels=corr_matrix.columns)

for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        text = plt.text(j, i, f"{corr_matrix.iloc[i, j]:.2f}",
                        ha="center", va="center", fontsize=8,
                        color="white" if abs(corr_matrix.iloc[i, j]) >
0.5 else "black")

plt.title('所有变量间的 Pearson 相关系数热力图', fontsize=14)
plt.tight_layout()
plt.show()

print("离群值检测报告 (IQR 方法)")
df_winsorized = df.copy()
numeric_cols = df.select_dtypes(include=[np.number]).columns
outliers_summary = []
print("列名 | 离群值数量 | 占比(%) | 下界 | 上界 | ")
print("-" * 70)

for col in numeric_cols:
    if col in ['chas', 'rad']:
        continue
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 3 * IQR
    upper = Q3 + 3 * IQR
    outliers_mask = (df[col] < lower) | (df[col] > upper)
    outlier_count = outliers_mask.sum()
    outlier_percent = outlier_count / len(df) * 100
    if outlier_count > 0:
        df_winsorized.loc[df[col] < lower, col] = lower
        df_winsorized.loc[df[col] > upper, col] = upper
        outliers_summary.append((col, outlier_count, outlier_percent,
lower, upper, outlier_count))
        print(f"{col:15} {outlier_count:8} {outlier_percent:8.2f}
{lower:8.2f} {upper:8.2f} ")
    else:
        print(f"{col:15} {outlier_count:8} {outlier_percent:8.2f}
{lower:8.2f} {upper:8.2f} ")

if outliers_summary:
    total_outliers = sum([count for _, count, _, _, _, _ in
outliers_summary])
    total_points = len(df) * len(numeric_cols)
    print(f"\n 处理汇总")
    print(f"检测到离群值的列数:
{len(outliers_summary)}/{len(numeric_cols)}")
    print(f"总计离群值数量: {total_outliers}")
```

```
    print(f"离群值占比: {total_outliers / total_points * 100:.2f}%")

X = df_winsorized.drop('medv', axis=1)
y = df_winsorized['medv']

X_train_full, X_test_final, y_train_full, y_test_final =
train_test_split(
    X, y, test_size=0.2, random_state=34
)
print(f"\n数据划分:")
print(f"   训练+验证集大小: {X_train_full.shape[0]} 条")
print(f"   独立测试集大小: {X_test_final.shape[0]} 条")

columns_to_scale = X_train_full.columns.drop(['chas', 'rad'])
scaler = StandardScaler()

X_train_full_scaled = X_train_full.copy()
X_train_full_scaled[columns_to_scale] =
scaler.fit_transform(X_train_full[columns_to_scale])

X_test_final_scaled = X_test_final.copy()
X_test_final_scaled[columns_to_scale] =
scaler.transform(X_test_final[columns_to_scale])

print(f"\n标准化完成: {len(columns_to_scale)} 个连续特征")
print(f"   标准化特征: {list(columns_to_scale)}")

alphas = np.logspace(-3, 3, 50)
ridge = Ridge(random_state=42)
kf = KFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(
    estimator=ridge,
    param_grid={'alpha': alphas},
    cv=kf,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    verbose=0
)

print("\n正在执行 5 折交叉验证...")
grid_search.fit(X_train_full_scaled, y_train_full)

best_alpha_cv = grid_search.best_params_['alpha']
best_mse_cv = -grid_search.best_score_
print(f"最优 alpha = {best_alpha_cv:.4f}")
print(f"平均验证 MSE = {best_mse_cv:.4f}")

final_model = Ridge(alpha=best_alpha_cv, random_state=42)
final_model.fit(X_train_full_scaled, y_train_full)

y_train_pred = final_model.predict(X_train_full_scaled)
train_mse = mean_squared_error(y_train_full, y_train_pred)
train_r2 = r2_score(y_train_full, y_train_pred)
```

```
print(f"\n 训练+验证集性能:")
print(f"   MSE = {train_mse:.4f}")
print(f"   R² = {train_r2:.4f}")

y_test_pred = final_model.predict(X_test_final_scaled)
test_mse = mean_squared_error(y_test_final, y_test_pred)
test_r2 = r2_score(y_test_final, y_test_pred)

print(f"\n 测试集性能:")
print(f"   MSE = {test_mse:.4f}")
print(f"   R² = {test_r2:.4f}")

cv_mse_scores = []
for alpha in alphas:
    model = Ridge(alpha=alpha, random_state=42)
    scores = cross_val_score(
        model, X_train_full_scaled, y_train_full,
        cv=kf, scoring='neg_mean_squared_error'
    )
    cv_mse_scores.append(-scores)
cv_mse_scores = np.array(cv_mse_scores)

mean_mse = cv_mse_scores.mean(axis=1)
std_mse = cv_mse_scores.std(axis=1)
best_idx = np.argmin(mean_mse)

plt.figure(figsize=(10, 6))
for i in range(5):
    plt.plot(alphas, cv_mse_scores[:, i], 'gray', alpha=0.4,
linewidth=1)
plt.plot(alphas, mean_mse, 'r-', linewidth=2.5, label=f'平均验证 MSE')
plt.fill_between(alphas, mean_mse - std_mse, mean_mse + std_mse,
color='red', alpha=0.1)
plt.axvline(alphas[best_idx], color='k', linestyle='--', alpha=0.8)

plt.xscale('log')
plt.xlabel('正则化系数 α (log scale)')
plt.ylabel('验证集 MSE')
plt.title('5 折交叉验证: α 参数搜索')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

coef_df = pd.DataFrame({
    'feature': X_train_full_scaled.columns,
    'coefficient': final_model.coef_,
    'abs_coef': np.abs(final_model.coef_)
}).sort_values('abs_coef', ascending=False)

intercept_row = pd.DataFrame({
    'feature': ['intercept'],
    'coefficient': [final_model.intercept_],
    'abs_coef': [np.abs(final_model.intercept_)]
})
```

```
coef_df = pd.concat([coef_df, intercept_row], ignore_index=True)

print(f"\n模型系数:")
print(coef_df[['feature',
'coefficient']].round(4).to_string(index=False))

residuals = y_test_final - y_test_pred

plt.figure(figsize=(8, 6))
plt.scatter(y_test_pred, residuals, alpha=0.7, edgecolors='k', s=50)
plt.axhline(y=0, color='red', linestyle='--', linewidth=1.5)
plt.xlabel('预测值 (Predicted medv)')
plt.ylabel('残差 (Residuals)')
plt.title('残差图')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"\n残差统计:")
print(f"   均值: {np.mean(residuals):.6f}")
print(f"   标准差: {np.std(residuals):.4f}")
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedShuffleSplit
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

filename = r"C:\Users\29414\Desktop\数据挖掘作业\波士顿
\boston_housing.csv"
df = pd.read_csv(filename)

print("缺失值检查")
if df.isnull().sum().sum() == 0:
    print("无缺失值")
    print(f"数据形状: {df.shape}")

print("\n离群值处理")
df_clean = df.copy()
numeric_cols = df.select_dtypes(include=[np.number]).columns

for col in numeric_cols:
    if col in ['chas', 'rad']:
        continue
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
```

```
    lower = Q1 - 3 * IQR
    upper = Q3 + 3 * IQR
    df_clean[col] = np.clip(df[col], lower, upper)

print("离群值已处理")

X = df_clean.drop('medv', axis=1)
y = df_clean['medv']

sss = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=34)
train_idx, test_idx = next(sss.split(X, X['chas']))

X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

print(f"\n数据划分:")
print(f"   训练集: {X_train.shape[0]} 条")
print(f"   测试集: {X_test.shape[0]} 条")
print(f"   训练集 chas=1: {X_train['chas'].mean():.2%}")
print(f"   测试集 chas=1: {X_test['chas'].mean():.2%}")

tree = DecisionTreeRegressor(random_state=42)

param_grid = {
    'max_depth': [3, 4, 5, 6, 7, 8, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'max_features': [None, 'sqrt', 'log2']
}

print("\n执行超参数调优...")
grid_search = GridSearchCV(
    estimator=tree,
    param_grid=param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    verbose=0
)

grid_search.fit(X_train, y_train)

best_tree = grid_search.best_estimator_
best_params = grid_search.best_params_
best_mse_cv = -grid_search.best_score_

print(f"最优参数: {best_params}")
print(f"5 折 CV 平均 MSE: {best_mse_cv:.4f}")

y_pred = best_tree.predict(X_test)
test_mse = mean_squared_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)

y_train_pred = best_tree.predict(X_train)
train_mse = mean_squared_error(y_train, y_train_pred)
```

```python
train_r2 = r2_score(y_train, y_train_pred)

print(f"训练集 MSE = {train_mse:.4f}")
print(f"训练集 R² = {train_r2:.4f}")

print(f"\n测试集性能:")
print(f"   MSE = {test_mse:.4f}")
print(f"   R² = {test_r2:.4f}")

residuals = y_test - y_pred
plt.figure(figsize=(8, 5))
plt.scatter(y_pred, residuals, alpha=0.7, edgecolors='k')
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('预测值')
plt.ylabel('残差')
plt.title('CART 决策树残差图')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

importances = best_tree.feature_importances_
features = X.columns
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10, 6))
plt.bar(range(len(importances)), importances[indices], align='center')
plt.xticks(range(len(importances)), [features[i] for i in indices],
rotation=45, ha='right')
plt.title('特征重要性')
plt.ylabel('重要性得分')
plt.tight_layout()
plt.show()

print("\n特征重要性排序:")
for i in indices:
    print(f"{features[i]:15}: {importances[i]:.4f}")
```